

Research Notes on Vanishing gradient problem

Vanishing gradient problem

>> Section 1

$$\Delta \theta = -\eta \cdot [\nabla \times L(x^T) (\nabla \theta F(x_{t-1}, u_t, \theta) + \nabla \times F(x_{t-1}, u_t, \theta) \nabla \theta F(x_{t-2}, u_{t-1}, \theta) + \dots)]^T$$
$$\{\displaystyle \Delta \theta = -\eta \cdot \left[\nabla_{\theta} F(x_{t-1}, u_t, \theta) + \nabla_{\theta} F(x_{t-2}, u_{t-1}, \theta) + \dots \right]^T$$
 where η is the learning rate. The vanishing/exploding gradient problem appears because there are repeated multiplications, of the form $\nabla \times F(x_{t-1}, u_t, \theta) \nabla \times F(x_{t-2}, u_{t-1}, \theta) \nabla \times F(x_{t-3}, u_{t-2}, \theta) \dots$

>> Section 2

$$dx_t = \nabla \theta F(x_{t-1}, u_t, \theta) d\theta + \nabla \times F(x_{t-1}, u_t, \theta) dx_{t-1}$$
$$dx_{t-1} = \nabla \theta F(x_{t-1}, u_t, \theta) d\theta + \nabla \times F(x_{t-1}, u_t, \theta) [\nabla \theta F(x_{t-2}, u_{t-1}, \theta) d\theta + \nabla \times F(x_{t-2}, u_{t-1}, \theta) dx_{t-2}]$$
$$\begin{aligned} dx_t &= [\nabla \theta F(x_{t-1}, u_t, \theta) + \nabla \times F(x_{t-1}, u_t, \theta) \nabla \theta F(x_{t-2}, u_{t-1}, \theta) + \dots] d\theta \\ &\quad + \nabla_{\theta} F(x_{t-1}, u_t, \theta) dx_{t-1} + \nabla_{\theta} F(x_{t-2}, u_{t-1}, \theta) dx_{t-2} + \dots \end{aligned}$$

>> Section 3

RNN For recurrent neural networks, the long short-term memory (LSTM) network was designed to solve the problem (Hochreiter & Schmidhuber, 1997). For the exploding gradient problem, (Pascanu et al, 2012) recommended gradient clipping, meaning dividing the gradient vector g by $\|g\| / g_{\max}$ if $\|g\| > g_{\max}$. This restricts the gradient vectors within a ball of radius $g_{\max}$.

>> Section 4

Faster hardware Hardware advances have meant that from 1991 to 2015, computer power (especially as delivered by GPUs) has increased around a million-fold, making standard backpropagation feasible for networks several layers deeper than when the vanishing gradient problem was recognized. Schmidhuber notes that this "is

basically what is winning many of the image recognition competitions now", but that it "does not really overcome the problem in a fundamental way" since the original models tackling the vanishing gradient problem by Hinton and others were trained in a Xeon processor, not GPUs.

>> Section 5

$$\nabla \times F(x_{t-1}, u_t, \theta) \nabla \times F(x_{t-2}, u_{t-1}, \theta) \dots \nabla \times F(x_{t-k}, u_{t-k+1}, \theta) = W \text{rec diag}(\sigma'(x_{t-1})) W \text{rec diag}(\sigma'(x_{t-2})) \dots W \text{rec diag}(\sigma'(x_{t-k}))$$
$$\begin{aligned} \nabla_{\theta} F(x_{t-1}, u_t, \theta) \nabla_{\theta} F(x_{t-2}, u_{t-1}, \theta) \dots \nabla_{\theta} F(x_{t-k}, u_{t-k+1}, \theta) \\ = W_{\text{rec}} \text{diag}(\sigma'(x_{t-1})) W_{\text{rec}} \text{diag}(\sigma'(x_{t-2})) \dots W_{\text{rec}} \text{diag}(\sigma'(x_{t-k})) \end{aligned}$$

>> Section 6

$$x_t = F(x_{t-1}, u_t, \theta) = W \text{rec} \sigma(x_{t-1}) + W \text{in} u_t + b$$
$$x_t = W_{\text{rec}} \sigma(x_{t-1}) + W_{\text{in}} u_t + b$$
 where $\theta = (W_{\text{rec}}, W_{\text{in}})$ is the network parameter, σ is the sigmoid activation function, applied to each vector coordinate separately, and b is the bias vector. Then, $\nabla \times F(x_{t-1}, u_t, \theta) = W \text{rec diag}(\sigma'(x_{t-1})) \nabla_{\theta} F(x_{t-1}, u_t, \theta) = W_{\text{rec}} \text{diag}(\sigma'(x_{t-1}))$, and so

>> Section 7

Recurrent network model A generic recurrent network has hidden states $h_1, h_2, \dots$, inputs $u_1, u_2, \dots$, and outputs $x_1, x_2, \dots$. Let it be parameterized by θ , so that the system evolves as

>> Section 8

Training the network requires us to define a loss function to be minimized. Let it be $L(x^T, u_1, \dots, u_T)$, then minimizing it by gradient descent gives

>> Section 9

Multi-level hierarchy In multi-level hierarchy of networks (Schmidhuber, 1992), pre-trained one level at a time through unsupervised learning, fine-tuned through backpropagation. Here each level learns a compressed representation of the observations that is fed to the next level.

Research Notes on Vanishing gradient problem

Vanishing gradient problem

>> Section 10

$$x_{t+1} = (1 - \epsilon) x_t + \epsilon \sigma(w x_t + b) + \epsilon w' u_t$$
$$\{ \displaystyle x_{t+1} = (1 - \epsilon) x_t + \epsilon \sigma(w x_t + b) + \epsilon w' u_t \}$$

>> Section 11

In machine learning, the vanishing gradient problem is the problem of greatly diverging gradient magnitudes between earlier and later layers encountered when training neural networks with backpropagation. In such methods, neural network weights are updated proportional to their partial derivative of the loss function. As the number of forward propagation steps in a network increases, for instance due to greater network depth, the gradients of earlier weights are calculated with increasingly many multiplications. These multiplications shrink the gradient magnitude. Consequently, the gradients of earlier weights will be exponentially smaller than the gradients of later weights. This difference in gradient magnitude might introduce instability in the training process, slow it, or halt it entirely. For instance, consider the hyperbolic tangent activation function. The gradients of this function are in range [0,1]. The product of repeated multiplication with such gradients decreases exponentially. The inverse problem, when weight gradients at earlier layers get exponentially larger, is called the exploding gradient problem. Backpropagation allowed researchers to train supervised deep artificial neural networks from scratch, initially with little success. Hochreiter's diplom thesis of 1991 formally identified the reason for this failure in the "vanishing gradient problem", which not only affects many-layered feedforward networks, but also recurrent networks. The latter are trained by unfolding them into very deep feedforward networks, where a new layer is created for each time-step of an input sequence processed by the network (the combination of unfolding and backpropagation is termed backpropagation through time).